

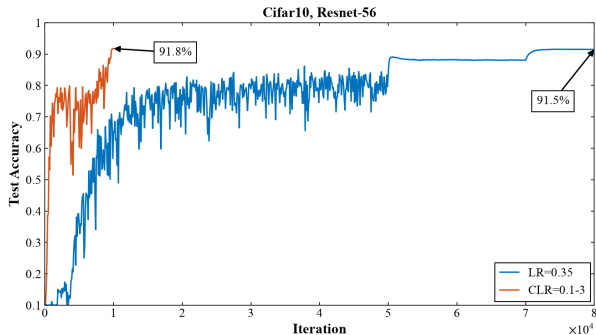
Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates

Leslie N. Smith, Nicholay Topin, [arXiv:1708.07120](https://arxiv.org/abs/1708.07120)

HW2, Efficient Models course, ITMO 2026

The claim

- ▶ ResNet-56 on CIFAR-10, batch 1000
- ▶ Typical training: 80,000 iterations, 91.2 %
- ▶ One cycle of a large learning rate: 10,000 iterations, 92.4 %
- ▶ The authors call this **super-convergence**: the same or better accuracy with an order of magnitude fewer iterations



Our reproduction of Fig. 1a

Two learning rate schedules

PC-LR, piecewise constant, the “typical” baseline: LR 0.35, divided by 10 at iterations 50k and 70k.

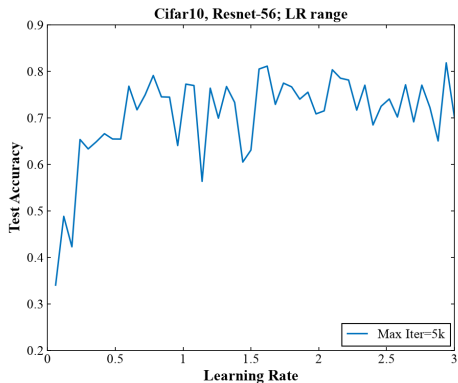
CLR, cyclical learning rate: LR goes linearly from `base_lr` to `max_lr` over `stepsize` iterations and back

$$\text{lr}(t) = \text{base} + (\text{max} - \text{base}) \cdot \max(0, 1 - |t/s - 2c - 1|), \quad c = \lfloor t/2s \rfloor, \quad s = \text{stepsize}$$

- ▶ Super-convergence uses one cycle, $0.1 \rightarrow 3 \rightarrow 0.1$, stepsize 5,000
- ▶ LR 3 is about 30 times larger than usual for this network
- ▶ The paper's **1cycle** policy: one cycle shorter than the run, then LR decays far below `base_lr`

LR range test: how large can the LR be?

- ▶ One short run, LR grows linearly from 0
- ▶ Usually test accuracy peaks and falls: the peak gives `max_lr`
- ▶ ResNet-56 on CIFAR-10 has no drop up to LR 3: accuracy stays at 0.7–0.8
- ▶ The paper reads this as a sign that super-convergence is possible



Our reproduction of Fig. 2b

Why it works: large learning rates regularize

- ▶ With a large LR, SGD can't settle in sharp minima and ends in flat, wide ones, which generalize better
- ▶ A large LR is regularization, so other regularization has to go down: less weight decay, less dropout, larger batches
- ▶ The paper's rule: **the amount of regularization must be balanced for each dataset and architecture**
- ▶ Running BN statistics have to keep up with fast-changing weights: BN moving average fraction 0.95 for CLR instead of 0.999
- ▶ The gain over PC-LR grows when labeled data is scarce: 80.6 vs 71.4 % on 10,000 training images (Table 1)

Estimating the optimal learning rate (Section 4)

Hessian-free view: the curvature that matters is along the SGD step. A finite difference of two gradients gives (AdaSecant)

$$\epsilon^* \approx \frac{\theta_{i+1} - \theta_i}{\nabla f(\theta_{i+1}) - \nabla f(\theta_i)} \Rightarrow \epsilon^* = \epsilon \frac{\theta_{i+1} - \theta_i}{2\theta_{i+1} - \theta_i - \theta_{i+2}}$$

- ▶ Only weights of three consecutive iterations are needed
- ▶ One global estimate: sum absolute values of numerator and denominator over all weights
- ▶ For ResNet-56 the estimate is 2–6 in the first 300 iterations and remains high with CLR: small curvature, flat minimum
- ▶ The paper uses it as an illustration, not as a training method

Results in the paper

Samples	Policy	Accuracy, %
50,000	PC-LR 0.35, 80k it.	91.2
50,000	CLR 0.1–3, 10k it.	92.4
40,000	PC-LR / CLR	89.1 / 91.1
30,000	PC-LR / CLR	85.7 / 89.6
20,000	PC-LR / CLR	82.7 / 87.9
10,000	PC-LR / CLR	71.4 / 80.6

Fig. 1a and Table 1, ResNet-56, CIFAR-10

Also shown in the paper:

- ▶ CIFAR-100, MNIST, ImageNet
- ▶ Wide ResNet, DenseNet, ResNet-50, Inception-ResNet-v2, ResNet-20/110
- ▶ Larger batches give a small improvement
- ▶ Adaptive optimizers (Nesterov, Adam, AdaDelta, AdaGrad) don't reach it

Reproduction setup

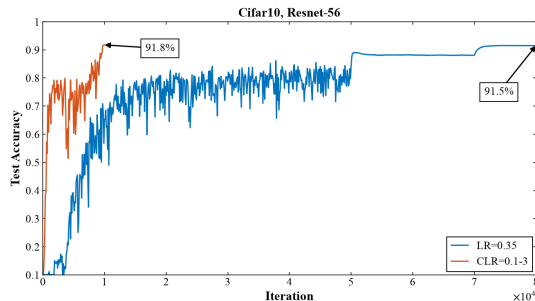
The official repo has the Caffe prototxt files and the authors' training logs. Following them:

- ▶ ResNet-56 from `Resnet56Cifar.prototxt`, FP32, PyTorch + Lightning
- ▶ Batch $1000 = 8 \text{ GPUs} \times 125$: BN normalizes every 125 images separately, running stats from the first GPU
- ▶ Caffe SGD: LR inside momentum, weight decay 10^{-4} on all parameters
- ▶ No shuffling, only random flips, mean image subtracted
- ▶ Test every 100 iterations on 25,000 images, wrapping around the test set as Caffe does
- ▶ Kaggle, $2 \times \text{T4}$, one process per GPU, same results as on one GPU

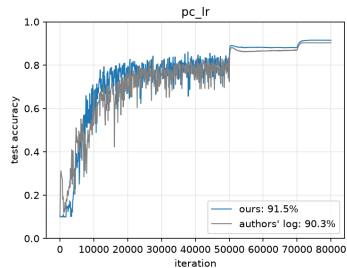
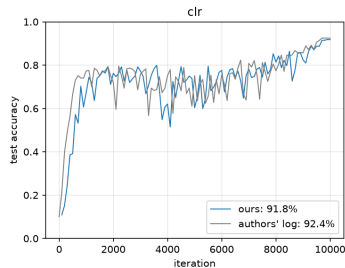
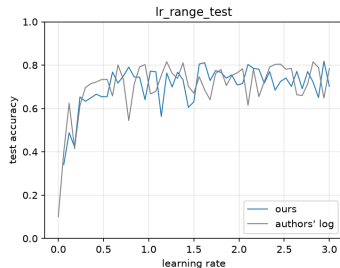
Results

Run	Paper	Authors' log	Ours
CLR, 10k it.	92.4	92.4	91.8
PC-LR, 80k it.	91.2	90.3	91.5

- ▶ CLR: 0.92 h, PC-LR: 7.40 h on $2 \times$ T4
- ▶ PC-LR reaches 90 % only at iteration 70,500
- ▶ Range test: 0.7–0.8 up to LR 3, as in the paper



Comparison with the authors' logs



- ▶ Curves follow the authors' Caffe runs within test-to-test noise
- ▶ Their own PC-LR log ends 0.9 points below the paper: differences of this size show up between runs

Conclusions

- ▶ Reproduced: one LR cycle up to 3 matches 80,000 iterations of typical training in 10,000 iterations, 8 times less GPU time
- ▶ Reproduced: the LR range test stays flat up to LR 3
- ▶ Not reproduced for lack of GPU time: the reduced training set runs of Table 1 (the 10,000-sample ones are implemented), other stepsizes, the LR estimate of Section 4, other datasets and architectures
- ▶ Code, results and report: <https://github.com/PE51K/itmo-ai-talent-hub-efficient-models-course/tree/main/hw2>